Multi-Agent Simulation Engine — operator behaviour & conjunction negotiation

SPEC-SW-006 v0.1 DRAFT Jeremie Nunez

1 Context

Thalamus currently answers *what is* and *what just happened* in the SSA knowledge graph. It cannot answer *what would happen if*. This spec introduces a multi-agent simulation engine, hosted in `packages/sweep`, that instantiates persistent agents from KG state, runs them through a turn-based loop, and produces a narrative report plus a deep-interaction chat surface.

Inspiration: MiroFish's five-stage pipeline (*graph build* → *environment setup* → *simulation* → *report* → *deep interaction*). This spec adapts that pattern to SSA, reusing the existing cortex registry, LLM transport, pgvector store, and BullMQ queues rather than running a Python sidecar.

Core idea — the swarm, not the sim

The primary unit of value is NOT a single high-fidelity simulation. It is a **swarm of cheap small-model "fish"** that collectively cover the space of plausible outcomes. Each fish is a short, nano-model-driven mini-sim with a perturbed seed (different god-event, different initial assumption, different persona tuning). The aggregator clusters outcomes and produces a distribution — "in 62% of futures, operator A maneuvers; in 28%, B maneuvers; in 10%, negotiation fails and escalates".

A single run is just a swarm of size 1. Determinism guarantees mean fixture mode can replay the full swarm byte-identically.

Two use cases are in-scope for v0:

- **UC1 — Operator behaviour swarm.** N agents (one per operator), run for a short compressed horizon, across K fish ($K \approx 50$–100) each with a different god-event perturbation (regulation variants, ASAT event on different satellites, launch surges at different scales, debris cascade severities). Output: a coverage report over operator decisions — "under these perturbations, what fraction of futures trigger a Starlink slot-share reshuffle?".
- **UC3 — Conjunction negotiation swarm.** A 2-agent swarm triggered by a conjunction finding. K fish ($K \approx 30$–50) each perturb the seed (delta-v budget, risk tolerance, insurance-loss assumption). Output: a distribution over resolutions (who maneuvers, split ratio) plus a confidence band. Feeds the modal resolution back into `finding-routing` as a `sweep_suggestion`, annotated with the distribution for reviewer context.

A fish runs on the nano model (`gpt-5.4-nano` via `callNano`) by default; swarm cost is $\sim$ \$0.01/fish, so $K = 100$ fish costs about \$1.

2 Goals

- **Swarm-first**: the orchestrator's primary API is `launchSwarm`; a single sim is $K = 1$. Each fish is a cheap nano-driven mini-sim with a perturbed seed.
- **Coverage over fidelity**: prefer 100 short cheap fish to 1 long expensive sim. The aggregator produces a distribution of outcomes, not a verdict.
- Model long-lived **agents within each fish** (not one-shot cortex calls) with stable identity, persona, and memory across turns.

- Drive **fish execution** over BullMQ with parallel fan-out (concurrency 8–16) so a 100-fish swarm finishes in < 2 minutes wall-clock on fixtures.
- Persist **per-agent memory** in pgvector scoped by `(sim_run_id, agent_id)` so fish cannot bleed into each other.
- Reuse existing LLM pathways: `callNano` (nano default, cheap) for fish bodies; `createLlmTransportWithMode` (Kimi K2) for the aggregator / reporter.
- Reuse the **cortex registry** as the per-turn reasoning primitive — no new LLM abstraction.
- Produce a **swarm coverage report** via a new `sim_swarm_reporter` cortex; a single-fish report via `sim_reporter`.
- Wire **UC3** into `finding-routing` so conjunction findings can automatically spawn a negotiation *swarm* whose modal resolution feeds back as a sweep suggestion, annotated with the full distribution for reviewer context.

3 Non-Goals

Non-Goal

This spec does NOT re-implement orbital propagation, does NOT compute real delta-v budgets, and does NOT produce legally-actionable maneuver plans. All physics is heuristic: mass-based cost bands (already available via `queryReplacementCost`) and fleet-level proxies. Realism is narrative, not numerical.

Non-Goal

This spec does NOT introduce streaming LLM calls inside the sim loop. Each turn is a single `await` against `LlmChatTransport`; streaming remains confined to the existing chat surface.

Non-Goal

This spec does NOT require MiroFish's Python runtime, Zep Cloud, or OASIS. The only runtime additions are (a) two new Postgres tables and (b) two new BullMQ workers.

4 Invariants

Invariant

Agents are append-only. An agent's `agent.id` persists across sim runs; `agent_memory` rows are never mutated, only appended and soft-retired via `retired_at`.

Invariant

A **turn** is the atomic unit of sim progress. Partial turns are rolled back: if any agent's turn call fails, the entire turn is discarded and BullMQ retries with exponential backoff (max 3 attempts).

Invariant

God-view injections (policy, event) are themselves `sim_turn` rows with `actor_kind = 'god'` — they live in the same timeline as agent turns so the narrative report can interleave them.

Invariant

No cross-sim memory bleed. An agent bound to operator `X` in `sim_run A` cannot read memory rows written in `sim_run B`. Memory is scoped by `(agent_id, sim_run_id)`.

Invariant

UC3 negotiation swarms produce at most one `sweep_suggestion` per swarm (not per fish), tagged with `sim_swarm_id`. The suggestion payload carries the modal resolution plus the full outcome distribution as metadata. Individual fish never emit suggestions directly.

Invariant

A swarm is deterministic under fixed `(baseSeed, perturbationSet, size, llmMode="fixtures")`. Re-running the same swarm config yields byte-identical fish outcomes and byte-identical aggregator output.

Invariant

Fish within a swarm are *independent* — no fish reads another fish's memory or timeline. Cross-fish aggregation happens only in the aggregator, which reads `sim_turn` rows grouped by `sim_run_id`. This guarantees embarrassingly-parallel fan-out.

Invariant

Swarm abort is fail-soft: if f of K fish fail (LLM error, timeout), the aggregator proceeds when $K - f \geq \lceil 0.8K \rceil$ (80% quorum). Below quorum, the swarm transitions to `failed` and no suggestion is emitted. Per-fish timeout is 60s.

Invariant

Thalamus never imports sweep. All new code lives in `packages/sweep/src/sim/`; the only Thalamus surfaces consumed are `CortexExecutor.runSkillFreeform`, `ThalamusDAGExecutor`, `createLlmTransportWithMode`, and the existing embedding helper. The two new cortex skills (`sim_operator_agent`, `sim_reporter`) live in Thalamus and are consumed from sweep through the standard cortex registry — no special coupling.

Invariant

`sim_turn` is the canonical, mode-agnostic timeline. Both drivers (DAG and Sequential) write every agent action and every god event there. The reporter and the chat surface query `sim_turn` exclusively — never the DAG driver's transient `research_finding` rows — so the two modes are observationally indistinguishable.

Invariant

Promotion of a `sim_turn` row to a `research_finding` is action-kind gated: only `maneuver`, `launch`, `retire` are promoted. Negotiation micro-actions (`propose_split`, `accept`, `reject`, `hold`, `lobby`) MUST NOT create `research_finding` rows. Rationale: keep the KG signal-rich; negotiation noise stays scoped to the sim run.

5 Architecture

5.1 Swarm layer (top)

The swarm is the public API. It fans out K fish over BullMQ, each fish is one `sim_run`, each fish picks a driver (DAG or Sequential) by its kind. The aggregator reads `sim_turn` across all fish in the swarm and emits a coverage report.

```
// swarm.service.ts
launchSwarm({
  kind: "uc1_operator_behavior" | "uc3_conjunction",
  baseSeed: SeedRefs,                    // fleet, finding, horizon
  perturbations: PerturbationSpec[],     // K entries
  llmMode: "cloud" | "fixtures" | "record",
}): Promise<{ swarmId: number }>
```

Each `PerturbationSpec` is applied to the base seed to produce a fish-specific seed. Examples:

- `{ godEvent: { kind: "asat_event", targetSatelliteId: 42 } }`
- `{ constraintOverride: { agentId: 7, maxDeltaVBudgetPerSat: 50 } }` (tighter budget for one agent)
- `{ personaTweak: { agentId: 7, riskProfile: "aggressive" } }`
- `{ launchSurge: { regimeId: 3, extraSatellites: 100 } }`
- `{ noop: true }` (baseline control fish, always present as fish[0])

5.2 Two drivers, one fish

The sim engine ships with **two turn drivers** selected by `sim_run.kind`:

- **DAG driver** (UC1) — one turn = one `ThalamusDAGExecutor.execute()` call. Agents are parallel cortex nodes (`sim_operator_agent`) in the same level; a reconciler node settles observations. Each turn is a `research_cycle`, giving a full audit trail for free (findings, edges, embeddings). Optimal when N agents decide simultaneously.
- **Sequential driver** (UC3) — one turn = one direct `CortexExecutor.runSkillFreeform()` call against the same `sim_operator_agent` skill. Agents alternate (A → B → A ...), intermediate state (current offer) lives only in `sim_turn`, no `research_finding` noise. Optimal for tight negotiation back-and-forth.

Both drivers write to the same `sim_turn` table — the canonical, mode-agnostic timeline consumed by the reporter and the chat surface. The cortex skill `sim_operator_agent` is shared: one prompt, two invocation paths.

Research-finding promotion. Every agent action is written to `sim_turn`. A subset (`action.kind` ∈ `{maneuver, launch, retire}`) is additionally promoted to `research_finding` for KG-level audit (embedding, edges to satellite / operator). Negotiation micro-actions (`propose_split, accept, reject, hold, lobby`) remain in `sim_turn` only, so the KG stays signal-rich.

5.3 Package layout

```
packages/sweep/src/sim/
  types.ts                      // SimSwarm, SimRun, Agent, Turn, Memory
      , GodEvent, Perturbation
  schema.ts                     // Zod schemas for turn outputs +
      perturbations
  agent-builder.ts              // KG -> Agent (persona synthesis)
```

```
  memory.service.ts              // write/read agent_memory (pgvector,
      scoped)
  swarm.service.ts               // fan-out K fish, track quorum
  perturbation.ts                // generators + applicator (seed + spec
      -> fish seed)
  aggregator.service.ts          // cluster fish outcomes, compute
      distribution
  turn-runner-dag.ts             // UC1 fish driver: build DAG, runCycle,
       persist
  turn-runner-sequential.ts      // UC3 fish driver: direct
      runSkillFreeform
  god-channel.service.ts         // inject events (per-fish, pre-seeded
      via perturbation)
  sim-orchestrator.service.ts    // internal: schedules turns within a
      fish
  reporter.service.ts            // renders single-fish narrative
  swarm-reporter.service.ts      // renders swarm coverage report
  promote.ts                     // modal outcome -> research_finding +
      sweep_suggestion

packages/sweep/src/jobs/workers/
  sim-turn.worker.ts             // one agent-turn, picks driver by
      sim_run.kind
  swarm-fish.worker.ts           // one fish lifecycle (start -> drain
      turns -> close)
  swarm-aggregate.worker.ts      // fires when quorum reached ->
      aggregator + reporter

packages/sweep/src/routes/
  swarm.routes.ts                // POST /swarm/uc1, POST /swarm/uc3, GET
       /swarm/:id

packages/thalamus/src/cortices/skills/
  sim_operator_agent.md          // NEW: shared per-turn agent prompt (
      nano-friendly)
  sim_reconciler.md              // NEW: no-LLM, post-turn settler for
      DAG driver
  sim_reporter.md                // NEW: single-fish narrative
  sim_swarm_reporter.md          // NEW: swarm-level coverage report

packages/thalamus/src/cortices/
  sql-helpers.sim-context.ts     // querySimAgentContext(simRunId,
      agentId, turn)
  sql-helpers.sim-timeline.ts    // querySimTimeline(simRunId) for single
      -fish reporter
  sql-helpers.sim-swarm.ts       // querySwarmOutcomes(swarmId) for
      aggregator + reporter
```

5.4 Data model

Five tables in `packages/db-schema/src/schema/sim.ts`:

```
export const simSwarm = pgTable("sim_swarm", {
  id: bigserial("id", { mode: "number" }).primaryKey(),
  kind: text("kind").$type<"uc1_operator_behavior" | "uc3_conjunction"
      >().notNull(),
  title: text("title").notNull(),
  baseSeed: jsonb("base_seed").$type<SeedRefs>().notNull(),
```

```
  perturbations: jsonb("perturbations").$type<PerturbationSpec[]>().
      notNull(),
  size: integer("size").notNull(),              // = perturbations.length
  config: jsonb("config").$type<SwarmConfig>().notNull(),
  // { llmMode, quorumPct: 0.8, perFishTimeoutMs: 60000, fishConcurrency
      : 8 }
  status: text("status").$type<"pending"|"running"|"done"|"failed">().
      notNull(),
  outcomeReportFindingId: bigint("outcome_report_finding_id", { mode: "
      number" }),
  suggestionId: bigint("suggestion_id", { mode: "number" }), // UC3
      modal outcome
  startedAt: timestamp("started_at", { withTimezone: true }).defaultNow
      (),
  completedAt: timestamp("completed_at", { withTimezone: true }),
  createdBy: bigint("created_by", { mode: "number" }),
});

export const simRun = pgTable("sim_run", {
  id: bigserial("id", { mode: "number" }).primaryKey(),
  swarmId: bigint("swarm_id", { mode: "number" })
    .references(() => simSwarm.id, { onDelete: "cascade" }).notNull(),
  fishIndex: integer("fish_index").notNull(),  // 0..swarm.size - 1
  kind: text("kind").$type<"uc1_operator_behavior" | "uc3_conjunction"
      >().notNull(),
  title: text("title").notNull(),
  seedRefs: jsonb("seed_refs").$type<SeedRefs>().notNull(),
  // e.g. { operatorIds: [...], conjunctionFindingId?: number,
      horizonDays: 30 }
  config: jsonb("config").$type<SimConfig>().notNull(),
  // { turnsPerDay: 1, maxTurns: 90, llmMode: "fixtures"|"cloud", seed:
      42 }
  status: text("status").$type<"pending"|"running"|"paused"|"done"|"
      failed">().notNull(),
  startedAt: timestamp("started_at", { withTimezone: true }).defaultNow
      (),
  completedAt: timestamp("completed_at", { withTimezone: true }),
  reportFindingId: bigint("report_finding_id", { mode: "number" }),
  createdBy: bigint("created_by", { mode: "number" }),
});

export const simAgent = pgTable("sim_agent", {
  id: bigserial("id", { mode: "number" }).primaryKey(),
  simRunId: bigint("sim_run_id", { mode: "number" })
    .references(() => simRun.id, { onDelete: "cascade" }).notNull(),
  operatorId: bigint("operator_id", { mode: "number" })
    .references(() => operator.id),
  persona: text("persona").notNull(), // system-prompt fragment
  goals: jsonb("goals").$type<string[]>().notNull(),
  constraints: jsonb("constraints").$type<Record<string, unknown>>().
      notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow
      (),
});

export const simTurn = pgTable("sim_turn", {
  id: bigserial("id", { mode: "number" }).primaryKey(),
  simRunId: bigint("sim_run_id", { mode: "number" })
    .references(() => simRun.id, { onDelete: "cascade" }).notNull(),
```

```
  turnIndex: integer("turn_index").notNull(),
  actorKind: text("actor_kind").$type<"agent"|"god"|"system">().notNull
      (),
  agentId: bigint("agent_id", { mode: "number" })
    .references(() => simAgent.id),
  action: jsonb("action").$type<TurnAction>().notNull(),
  rationale: text("rationale").notNull(),
  observableSummary: text("observable_summary").notNull(),
  // what other agents see next turn; may omit private rationale
  llmCostUsd: numeric("llm_cost_usd", { precision: 10, scale: 6 }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow
      (),
}, (t) => ({
  uniqTurn: uniqueIndex("sim_turn_uniq").on(t.simRunId, t.turnIndex, t.
      agentId),
}));

export const simAgentMemory = pgTable("sim_agent_memory", {
  id: bigserial("id", { mode: "number" }).primaryKey(),
  simRunId: bigint("sim_run_id", { mode: "number" })
    .references(() => simRun.id, { onDelete: "cascade" }).notNull(),
  agentId: bigint("agent_id", { mode: "number" })
    .references(() => simAgent.id).notNull(),
  turnIndex: integer("turn_index").notNull(),
  kind: text("kind").$type<"self_action"|"observation"|"belief">().
      notNull(),
  content: text("content").notNull(),
  embedding: vector("embedding", { dimensions: EMBEDDING_DIMENSIONS }),
  retiredAt: timestamp("retired_at", { withTimezone: true }),
});
```

HNSW index on `sim_agent_memory.embedding` follows the existing pattern (`migrations/0001_hnsw_index.sql`

5.5 Turn loop — DAG driver (UC1)

```
// turn-runner-dag.ts --- one turn = one DAG cycle, N agents in parallel
async function runTurnDag(simRunId: number, turnIndex: number) {
  const agents = await loadAgents(simRunId);

  const dag: DAGPlan = {
    intent: `sim ${simRunId} turn ${turnIndex}`,
    complexity: "moderate",
    nodes: [
      ...agents.map((a) => ({
        cortex: "sim_operator_agent",
        params: { simRunId, agentId: a.id, turnIndex },
        dependsOn: [],
      })),
      {
        cortex: "sim_reconciler",   // settles observations, writes
            sim_turn rows
        params: { simRunId, turnIndex },
        dependsOn: agents.map((_, i) => `sim_operator_agent#${i}`),
      },
    ],
  };

  // Existing Thalamus plumbing: persists research_cycle, findings,
      edges
```

```
  await thalamusService.runCycle({ dag, mode: llmMode });

  // Reconciler already wrote sim_turn + memory; we just advance.
  await scheduleNext(simRunId, turnIndex + 1);
}
```

5.6 Turn loop — Sequential driver (UC3)

```
// turn-runner-sequential.ts --- one turn = one agent speaks, strict A->
   B alternation
async function runTurnSequential(simRunId: number, turnIndex: number) {
  const agent = pickSpeaker(simRunId, turnIndex); // alternates on
     turnIndex parity
  const ctx = await buildAgentContext(db, agent.id, turnIndex);

  const response = await cortexExecutor.runSkillFreeform({
    skillName: "sim_operator_agent",
    systemPrompt: agent.persona,
    userPrompt: renderTurnPrompt(ctx),
    schema: turnResponseSchema,
    mode: llmMode,
  });

  await db.transaction(async (tx) => {
    const turn = await insertSimTurn(tx, { simRunId, turnIndex, agent,
       response });
    await memory.write(tx, { self_action: turn });
    await memory.write(tx, { observation_for_others: turn.
       observableSummary });
    if (isKgPromotable(response.action)) {
      await promoteToFinding(tx, turn); // maneuver/launch/retire ->
         research_finding
    }
    if (isTerminal(response.action)) {
      await closeRun(tx, simRunId, turn); // accept / reject -> done
    }
  });

  await scheduleNext(simRunId, turnIndex + 1);
}
```

5.7 God channel

God events are injected via `POST /sim/:id/god` (admin-only). Payload is stored as a `sim_turn` row with `actor_kind = 'god'`. Agents see god events as part of their observable timeline — no special branch in the agent prompt.

5.8 UC3 wiring

`finding-routing.ts` gains a new hook: if a finding has `cortex = 'conjunction_analysis'` and `urgency >= 0.7`, enqueue a `sim-advance` job of kind `uc3_conjunction` with the finding id as seed. The sim runs to completion (max 20 turns), its resolution becomes a `sweep_suggestion` via `SweepRepository.enqueueSuggestion`, tagged with `sim_run_id`. Reviewer sees the suggestion in the normal inbox flow.

6 Interface

```
// sim-orchestrator.service.ts
export interface SimOrchestrator {
  startUc1(opts: {
    operatorIds: number[];
    horizonDays: number;
    turnsPerDay?: number;
    godEvents?: GodEventSeed[];
    mode?: "cloud" | "fixtures" | "record";
    createdBy?: number;
  }): Promise<{ simRunId: number }>;

  startUc3(opts: {
    conjunctionFindingId: number;
    maxTurns?: number;
    mode?: "cloud" | "fixtures" | "record";
  }): Promise<{ simRunId: number }>;

  pause(simRunId: number): Promise<void>;
  resume(simRunId: number): Promise<void>;
  inject(simRunId: number, event: GodEventSeed): Promise<void>;
  status(simRunId: number): Promise<SimStatus>;
}

// routes/sim.routes.ts
POST   /admin/sim/uc1          body: Uc1StartRequest
POST   /admin/sim/uc3          body: Uc3StartRequest
GET    /admin/sim/:id          -> SimStatus + last 20 turns
POST   /admin/sim/:id/god      body: GodEventSeed
POST   /admin/sim/:id/pause
POST   /admin/sim/:id/resume
GET    /admin/sim/:id/report   -> rendered narrative (from sim_reporter
    cortex)
```

7 Scenarios

Scenario: UC1 — Free-running operator sim, 3 operators, 30-day horizon

Given three operators (SpaceX, OneWeb, CNSA) each with ≥ 50 satellites in KG
And the orchestrator is booted with `mode = "fixtures"` for determinism
When `startUc1({ operatorIds: [1,2,3], horizonDays: 30, turnsPerDay: 1 })` is called
Then a `sim_run` row is created with `status = "running"`
And three `sim_agent` rows exist, one per operator, each with a persona synthesized from operator metadata + fleet stats
And the BullMQ `sim-turn` queue has exactly 3 jobs queued (turn 0 for each agent)
And after 90 turns (3 agents $\times$ 30 days), `status = "done"` and exactly 90 `sim_turn` rows exist.

Scenario: UC1 — God-view ASAT injection mid-run

Given a UC1 run is at turn 15 of 30, status `running`
When admin POSTs `/admin/sim/:id/god` with {`kind: "asat_event", targetSatelliteId: 42`}

Then one `sim_turn` row is appended with `actor_kind = "god"`
And on turn 16, every agent's observable context includes the god event
And at least one agent's turn-16 action references the ASAT event in its `rationale`.

Scenario: UC3 — Conjunction auto-spawn from finding

Given a conjunction finding is persisted with cortex `conjunction_analysis`, urgency = 0.85, and two satellites tied to operators A and B
When `finding-routing` processes the finding
Then a `sim_run` of kind `uc3_conjunction` is created within 2 seconds
And two `sim_agent` rows exist, one per operator
And within 60 seconds the run completes with $\leq$ 20 turns
And a `sweep_suggestion` is enqueued with `sim_run_id` set
And the suggestion's `actions` field includes a `maneuver` action targeting one of the two satellites.

Scenario: Fixture determinism

Given a UC3 run with `mode = "fixtures"`, `config.seed = 42`, fixed `conjunctionFindingId`
When the run is executed twice in succession
Then both runs produce byte-identical `sim_turn.action` and `rationale` for every turn
And both runs produce the same resolution `sweep_suggestion` payload.

Scenario: Turn failure is atomic

Given a UC1 run where the LLM transport throws on turn 7 for agent 2
When the worker catches the error
Then no `sim_turn` row is written for turn 7 agent 2
And no `sim_agent_memory` row is written for that turn
And BullMQ retries the job up to 3 times with exponential backoff
And after 3 failures, the run transitions to `status = "failed"` and no further turns are scheduled.

Scenario: Memory scoping

Given operator A participated in sim run X (20 turns of memory) and sim run Y (5 turns)
When agent-A in run Y queries its memory at turn 3
Then the vector search returns only rows where `sim_run_id = Y`
And zero rows from run X are returned even if semantically similar.

8 Acceptance criteria

Acceptance Criterion AC-1 Schema

Migration applies cleanly: `make db-migrate` produces `sim_run`, `sim_agent`, `sim_turn`, `sim_agent_memory` tables + HNSW index on `sim_agent_memory.embedding`. `pnpm typecheck` is 0 errors.

Acceptance Criterion AC-2 UC3 happy path

`make sim-uc3` (new Makefile target) completes in < 90s in fixtures mode, produces ≥ 4 turns, writes a `sweep_suggestion` tagged with `sim_run_id`, and its `reportFindingId` resolves to a `research_finding` row with non-empty summary.

Acceptance Criterion AC-3 UC1 happy path

`make sim-uc1` with 3 operators and `horizonDays = 5` completes in < 120s in fixtures mode, produces exactly 15 agent turns (ignoring god turns), and the final report mentions all 3 operators by name.

Acceptance Criterion AC-4 God injection

`POST /admin/sim/:id/god` during a running UC1 appends a god turn within 500ms; the next agent turn's rationale references the god event string.

Acceptance Criterion AC-5 Determinism

Two back-to-back fixture runs of UC3 with identical seed produce identical `action` JSON for every turn (byte-equal when sorted).

Acceptance Criterion AC-6 Isolation

Thalamus package continues to typecheck and pass its existing tests with zero imports from `@interview/sweep`.

Acceptance Criterion AC-7 Chat deep-interaction

The existing `satellite-sweep-chat` accepts a `simRunId` scope parameter; when set, its retrieval pulls from `sim_turn` + `sim_agent_memory` for the given run instead of the global KG.

9 Open questions

1. Should `observableSummary` be generated by the same LLM call as `action + rationale`, or by a second cheap pass (`gpt-5.4-nano`)? Two-pass is cleaner (prevents rationale leakage) but doubles cost.
2. For UC1 at scale ($N \geq 10$ agents, ≥ 100 days), should turns within a day run in parallel (all agents decide simultaneously, observations settle at end-of-day) or strictly sequential? Parallel is more realistic and faster; sequential is easier to debug.
3. UC3: if the two agents fail to converge within `maxTurns`, should the sim emit a "no-agreement" suggestion (escalate to human reviewer) or a "default: larger fleet maneuvers" suggestion? Current proposal: escalate.

10 References

- MiroFish README — https://github.com/666ghj/MiroFish/blob/main/README.md
- SPEC-SW-001 Nano Sweep (`nano-sweep.tex`)
- SPEC-SW-002 Finding Routing (`finding-routing.tex`) — UC3 integration point

- SPEC-SW-003 Resolution (`resolution.tex`) — consumes UC3 output
- SPEC-TH-* Cortex Pattern — `runSkillFreeform` is the per-turn primitive